

FULL STACK WEB DEVELOPMENT

JavaScript & Python-Django Track

Student Textbook — Beginner Edition

MODULE 1

MODULE 1: Introduction to HTML

HTML stands for HyperText Markup Language. It is the skeleton of every webpage you see on the internet. Just like a building needs a frame, every website needs HTML as its foundation.

In this module, you will learn how to create your first HTML document, understand its structure, and view it in a web browser.

1.1 Creating an HTML Document

An HTML document is simply a text file saved with the .html extension. You can create one using any text editor — Notepad (Windows), TextEdit (Mac), or a code editor like VS Code.

Steps to create your first HTML file:

1. Open your text editor (VS Code is recommended).
2. Click File > New File.
3. Type your HTML code.
4. Save the file as index.html.



WORKOUT

1. Download and install VS Code from <https://code.visualstudio.com>
2. Create a new file and save it as mypage.html on your Desktop.
3. Open the file in your browser by double-clicking it.
4. What do you see? (The page will be blank — that is expected!)

1.2 HTML Document Overview

Every HTML document follows a standard structure. Think of it like the format of a letter — it always has a greeting, a body, and a sign-off.

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first webpage.</p>
  </body>
</html>
```

Here is what each part means:

- `<!DOCTYPE html>` — Tells the browser this is an HTML5 document.
- `<html>` — The root element. Everything goes inside here.
- `<head>` — Contains behind-the-scenes information (title, links, meta data).
- `<title>` — The text shown on the browser tab.
- `<body>` — Everything visible on the page goes here.

**TIP**

Tags come in pairs: an opening tag `<tag>` and a closing tag `</tag>`.

The closing tag has a forward slash `/` before the tag name.

Content goes between the opening and closing tags.

**WORKOUT**

1. Copy the code above into your `mypage.html` file.
2. Change the `<title>` text to your own name.
3. Change the `<h1>` to say "Welcome to My Website".
4. Add a `<p>` tag below with a sentence describing yourself.
5. Save and refresh in the browser.

1.3 Viewing a Document in the Browser

After creating your HTML file, you need to open it in a web browser to see the result. There are two ways to do this:

5. Double-click the `.html` file — it opens in your default browser.
6. In VS Code, right-click the file and select "Open with Live Server" (requires the Live Server extension).

**TIP**

Install the "Live Server" extension in VS Code.

It auto-refreshes your browser every time you save the file.

This saves you from manually refreshing every time you make a change.

**WORKOUT**

1. Install the Live Server extension in VS Code.

2. Open your mypage.html with Live Server.
3. Make a change to the <h1> text and save.
4. Watch the browser update automatically!

MODULE 2: Text Formatting

Text is the most important content on any webpage. HTML provides many tags to format, organize, and style text. In this section you will learn how to control the appearance and meaning of text.

2.1 Basic HTML Tags

HTML tags are keywords surrounded by angle brackets `< >`. Most tags have an opening and a closing version.

```
<h1>Heading 1 – Largest</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6 – Smallest</h6>
<p>This is a paragraph of text.</p>
```



WORKOUT

1. Create a new file called `formatting.html`.
2. Add all six heading tags (`h1` through `h6`) with different text.
3. Add two `<p>` tags below with sentences.
4. Open in browser and observe the size difference between headings.

2.2 Bold and Italic Text

You can make text bold or italic to give it emphasis.

```
<b>This text is bold</b>
<strong>This text is also bold (and has meaning)</strong>

<i>This text is italic</i>
<em>This text is italic (with emphasis meaning)</em>
```



TIP

`<b>` and `<i>` are visual-only tags.
`<strong>` and `<em>` carry semantic meaning — screen readers use them.
Best practice: prefer `<strong>` over `<b>` and `<em>` over `<i>`.

 WORKOUT

1. In your `formatting.html`, add a paragraph.
2. Make one word bold using `<strong>`.
3. Make another word italic using `<em>`.
4. Try nesting them: `<strong><em>bold and italic</em></strong>`.

2.3 Strikeout, Superscript, and Subscript

```
<!-- Strikethrough -->
<s>This text is crossed out</s>
<del>This was deleted</del>

<!-- Superscript (above the line) -->
<p>Water formula: H<sup>2</sup>O</p>
<p>Math: 5<sup>2</sup> = 25</p>

<!-- Subscript (below the line) -->
<p>CO<sub>2</sub> is carbon dioxide</p>
```

 WORKOUT

1. Write a sentence and strike through one word.
2. Write the chemical formula for water: H₂O using `<sup>`.
3. Write the chemical formula for carbon dioxide: CO₂ using `<sub>`.
4. Show the math: x squared = x² using `<sup>`.

2.4 Special Characters

Some characters have special meaning in HTML (like `<` and `>`). To display them, use HTML entities.

```
&lt; → displays <
&gt; → displays >
&amp; → displays &
&copy; → displays ©
&nbsp; → non-breaking space
&quot; → displays "
```

 WORKOUT

1. Write a paragraph that includes a copyright symbol ©.
2. Display the text: <html> on your page (without it being a tag).
3. Display the text: "Tom & Jerry" using entities.

2.5 Block Quotes

Use <blockquote> when quoting from another source. Browsers indent it automatically.

```
<p>Albert Einstein once said:</p>
<blockquote>
  Imagination is more important than knowledge.
</blockquote>
```

WORKOUT

1. Write a page about your favourite quote.
2. Use a <p> to introduce the quote and who said it.
3. Place the actual quote inside <blockquote> tags.

2.6 Comments in HTML

Comments are notes you leave in your code. The browser ignores them completely. They help you and other developers understand what the code does.

```
<!-- This is a comment. The browser ignores this. -->

<!-- Navigation Section -->
<nav>
  <!-- Links will go here later -->
</nav>
```

2.7 Ordered and Unordered Lists

HTML has two main types of lists:

- — Unordered list (bullet points)
- — Ordered list (numbered)
- — List item (used inside both)

```
<!-- Unordered List -->
<ul>
```

```
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
</ul>

<!-- Ordered List -->
<ol>
  <li>Wake up</li>
  <li>Open VS Code</li>
  <li>Write HTML</li>
</ol>
```



WORKOUT

1. Create a shopping list using an unordered list (at least 5 items).
2. Create a "Steps to make breakfast" guide using an ordered list.
3. BONUS: Create a nested list — a list inside a list item.

2.8 Definition Lists

Definition lists are used for terms and their explanations — like a glossary.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language – the structure of web pages.</dd>

  <dt>CSS</dt>
  <dd>Cascading Style Sheets – controls the appearance of web pages.</dd>
</dl>
```



WORKOUT

1. Create a Web Development Glossary page.
2. Define at least 5 terms: HTML, CSS, JavaScript, Browser, Server.
3. Give a simple beginner-friendly definition for each.

2.9 Horizontal Lines

The `<hr>` tag draws a horizontal line to separate content. It is an empty tag — no closing tag needed.

```
<h2>Section 1</h2>
```



```
<p>Content for section 1.</p>
```

```
<hr>
```

```
<h2>Section 2</h2>
```

```
<p>Content for section 2.</p>
```

MODULE 3: Images

Images make webpages come alive. In this module, you will learn how to add images to your pages and make them accessible to all users.

3.1 Image Formats

Different image formats serve different purposes:

- **JPG/JPEG** — Best for photographs. Small file size, slight quality loss.
- **PNG** — Best for graphics, logos, images with transparent backgrounds.
- **GIF** — Supports animation. Limited to 256 colours.
- **WebP** — Modern format. Smaller files, high quality. Supported by most browsers.

3.2 Adding Images

The `<img>` tag is used to add images. It is a self-closing (empty) tag.

```
<!-- Basic image -->


<!-- Image with size -->


<!-- Image from the internet (absolute URL) -->

```

Key attributes:

- **src** — The path to the image file (required).
- **alt** — Description of the image (required for accessibility).
- **width / height** — Size of the image in pixels.

TIP

Always include the alt attribute on every image.

Screen readers (used by visually impaired users) read the alt text aloud.

If an image fails to load, the alt text is shown instead.

For decorative images, use `alt=""` (empty) to tell screen readers to skip it.

3.3 Absolute vs Relative URLs

There are two ways to reference an image (or any file):

- Absolute URL — The full web address. Example: `https://example.com/img/photo.jpg`
- Relative URL — A path relative to the current file. Example: `images/photo.jpg`

```
<!-- Absolute URL (from the internet) -->


<!-- Relative URL (file is in an images folder) -->


<!-- File is one folder up -->

```



WORKOUT

1. Create a folder called "images" next to your HTML file.
2. Download any image from the internet and save it in that folder.
3. Display the image on your page using a relative URL.
4. Add a meaningful alt attribute.
5. BONUS: Also display an image using an absolute URL from the web.

MODULE 4: General HTML Attributes

Attributes give extra information to HTML elements. Three of the most important are id, class, and data.

4.1 The id Attribute

The id attribute gives an element a unique name. No two elements on the same page should share the same id.

```
<h1 id="main-title">Welcome</h1>
<p id="intro-text">This is the introduction.</p>

<!-- Link to an element by id -->
<a href="#main-title">Go to Title</a>
```

4.2 The class Attribute

The class attribute is used to group elements together. Unlike id, multiple elements can share the same class. It is mainly used with CSS to style groups of elements.

```
<p class="highlight">This paragraph is highlighted.</p>
<p class="highlight">This one too!</p>
<p>This one is normal.</p>

<!-- An element can have multiple classes -->
<p class="highlight large-text">Big and highlighted!</p>
```

4.3 The data Attribute

The data-* attribute lets you store custom information inside HTML elements. You can name it anything after "data-". JavaScript can later read this information.

```
<button data-product-id="42" data-price="1500">Buy Now</button>

<li data-category="fruit">Apple</li>
<li data-category="vegetable">Carrot</li>
```



WORKOUT

1. Create a page with 3 paragraphs.
2. Give each paragraph a unique id.
3. Give two of them the same class name "featured".

4. Add a data-author="YourName" attribute to each paragraph.
5. BONUS: Create a navigation link that jumps to each paragraph by id.

MODULE 5: Links

Links (also called hyperlinks) are what make the web the web. They connect pages, websites, and resources together. The `<a>` tag is used to create links.

5.1 Link to Another Web Page

```
<!-- Link to an external website -->
<a href="https://www.google.com">Go to Google</a>

<!-- Link to another page in your own site -->
<a href="about.html">About Us</a>

<!-- Link to a page in a subfolder -->
<a href="pages/contact.html">Contact</a>
```

5.2 Link Within a Page (Anchor Links)

You can link to a specific section on the same page using ids.

```
<!-- The destination (must have an id) -->
<h2 id="about">About Section</h2>

<!-- The link that jumps to it -->
<a href="#about">Jump to About Section</a>
```

5.3 Link to Email

```
<a href="mailto:hello@example.com">Send us an email</a>

<!-- With subject and body pre-filled -->
<a href="mailto:hello@example.com?subject=Hello&body=I want to enquire...">
  Email Us
</a>
```

5.4 Link Target

The target attribute controls where the link opens.

```
<!-- Open in same tab (default) -->
<a href="https://google.com" target="_self">Google (same tab)</a>
```

```
<!-- Open in a new tab -->  
<a href="https://google.com" target="_blank">Google (new tab)</a>
```

**TIP**

Always use `target="_blank"` for external links.

This keeps users on your site while they browse the external one.

Add `rel="noopener noreferrer"` for security: `<a href="..." target="_blank" rel="noopener noreferrer">`

5.5 Link States (CSS Preview)

Links have four states that can be styled with CSS:

- `:link` — The default, unvisited state.
- `:visited` — After the link has been clicked.
- `:hover` — When the mouse is over the link.
- `:active` — The moment the link is being clicked.

**WORKOUT**

1. Create a navigation bar with 4 links: Home, About, Services, Contact.
2. Make the Home link go to `index.html`.
3. Make the Contact link open a mail to your email address.
4. Make the Services link open in a new tab.
5. Add a "Back to Top" link that jumps to the top of the page.

MODULE 6: Tables

Tables are used to display data in rows and columns — like a spreadsheet on a webpage.

6.1 Basic Table Structure

```
<table>
  <tr>                                <!-- table row -->
    <th>Name</th>                     <!-- table header -->
    <th>Age</th>
    <th>City</th>
  </tr>
  <tr>
    <td>Amaka</td> <!-- table data -->
    <td>22</td>
    <td>Lagos</td>
  </tr>
  <tr>
    <td>Emeka</td>
    <td>25</td>
    <td>Abuja</td>
  </tr>
</table>
```

Key table tags:

- `<table>` — Wraps the entire table.
- `<tr>` — A table row.
- `<th>` — A header cell (bold and centered by default).
- `<td>` — A data cell.
- `<caption>` — A title above the table.

6.2 Adding a Border and Caption

```
<table border="1">
  <caption>Student Results</caption>
  <tr>
    <th>Student</th>
    <th>Score</th>
    <th>Grade</th>
  </tr>
  <tr>
    <td>Chioma</td>
```



```
<td>85</td>
<td>A</td>
</tr>
</table>
```

6.3 Spanning Cells

Sometimes a cell needs to span across multiple columns or rows.

```
<!-- colspan: span across columns -->
<tr>
  <td colspan="2">This cell spans 2 columns</td>
</tr>

<!-- rowspan: span down rows -->
<tr>
  <td rowspan="2">This cell spans 2 rows</td>
  <td>Row 1, Col 2</td>
</tr>
<tr>
  <td>Row 2, Col 2</td>
</tr>
```

WORKOUT

1. Create a timetable for your week (Monday–Friday, morning/afternoon).
2. Use `<th>` for the days and time slots.
3. Use `colspan` to merge a cell that covers two periods (e.g., lunch break).
4. Add a `<caption>` that says "Weekly Timetable".
5. BONUS: Add a border using CSS (`border-collapse: collapse`).

MODULE 7: Media

HTML5 introduced built-in support for audio, video, and embedded content. You no longer need plugins like Flash!

7.1 Audio

```
<audio controls>
  <source src="music.mp3" type="audio/mpeg">
  <source src="music.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

Useful attributes:

- `controls` — Shows play, pause, volume controls.
- `autoplay` — Plays automatically when page loads.
- `loop` — Repeats the audio.
- `muted` — Starts muted.

7.2 Video

```
<video width="640" height="360" controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video element.
</video>
```

7.3 Iframe (Embedded Content)

An `iframe` embeds another webpage or resource inside your page. Common use: YouTube videos, Google Maps.

```
<!-- Embedding a YouTube video -->
<iframe
  width="560"
  height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  allowfullscreen>
</iframe>

<!-- Embedding Google Maps -->
```

```
<iframe
  src="https://maps.google.com/maps?q=Lagos&output=embed"
  width="600" height="400">
</iframe>
```

WORKOUT

1. Create a media page.
2. Embed a YouTube video of your choice using an iframe.
3. Embed a Google Map showing your city.
4. BONUS: Add an audio element with controls.

MODULE 8: Forms

Forms allow users to send data to a server — login pages, sign-up forms, contact forms, and checkouts all use HTML forms.

8.1 Basic Form Structure

```
<form action="/submit" method="POST">

  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name" placeholder="Enter your name">

  <label for="email">Email:</label>
  <input type="email" id="email" name="email">

  <button type="submit">Submit</button>

</form>
```

8.2 Input Types

The type attribute controls what kind of input is shown:

<code><input type="text"></code>	<code><!-- Regular text --></code>
<code><input type="email"></code>	<code><!-- Email address --></code>
<code><input type="password"></code>	<code><!-- Hidden text --></code>
<code><input type="number"></code>	<code><!-- Numbers only --></code>
<code><input type="date"></code>	<code><!-- Date picker --></code>
<code><input type="checkbox"></code>	<code><!-- Tick box --></code>
<code><input type="radio"></code>	<code><!-- Select one option --></code>
<code><input type="file"></code>	<code><!-- Upload a file --></code>
<code><input type="range"></code>	<code><!-- Slider --></code>
<code><input type="color"></code>	<code><!-- Color picker --></code>
<code><input type="submit"></code>	<code><!-- Submit button --></code>
<code><input type="reset"></code>	<code><!-- Reset form --></code>

8.3 Textarea, Select, and Fieldset

```
<!-- Multi-line text input -->
<textarea name="message" rows="5" cols="40"></textarea>
```

```
<!-- Dropdown select -->
<select name="country">
  <option value="ng">Nigeria</option>
  <option value="gh">Ghana</option>
  <option value="ke">Kenya</option>
</select>

<!-- Fieldset groups related inputs -->
<fieldset>
  <legend>Personal Info</legend>
  <input type="text" name="firstname" placeholder="First Name">
  <input type="text" name="lastname" placeholder="Last Name">
</fieldset>
```

8.4 Form Validation Attributes

HTML5 includes built-in validation so you do not need JavaScript for basic checks:

```
<!-- Required field -->
<input type="text" name="name" required>

<!-- Minimum and maximum length -->
<input type="text" name="username" minlength="3" maxlength="20">

<!-- Pattern (regular expression) -->
<input type="text" name="code" pattern="[A-Z]{3}" placeholder="3 uppercase letters">

<!-- Number range -->
<input type="number" name="age" min="1" max="120">
```

8.5 Form Method and Action

The form element has two key attributes:

- **action** — Where to send the form data (a URL or server endpoint).
- **method** — How to send it: GET (visible in URL) or POST (hidden, secure).

```
<!-- GET method (used for search forms) -->
<form action="/search" method="GET">
  <input type="text" name="q" placeholder="Search...">
  <button type="submit">Search</button>
</form>
```

```
<!-- POST method (used for login, registration) -->
<form action="/login" method="POST">
  <input type="email" name="email">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
```

WORKOUT

1. Build a Student Registration Form with these fields:
 - Full Name (text, required)
 - Email (email type, required)
 - Phone Number (tel type)
 - Gender (radio buttons: Male / Female)
 - Course (dropdown: HTML, CSS, JavaScript)
 - Date of Birth (date type)
 - Message/Bio (textarea)
 - Submit button
2. Add required validation to Name, Email, and Gender.
3. Group Personal Info and Course Info into separate fieldsets.

MODULE 2

MODULE 2: CSS — Cascading Style Sheets

CSS controls how your HTML looks — the colours, fonts, sizes, layouts, and animations. If HTML is the skeleton, CSS is the skin, hair, and clothes.

1. What is CSS?

CSS stands for Cascading Style Sheets. It is a language used to describe the presentation of a document written in HTML.

- Without CSS — pages look plain and unstyled.
- With CSS — pages look professional and beautiful.



TIP

"Cascading" means styles flow down from parent to child elements.

A style applied to `<body>` will affect all elements inside it, unless overridden.

2. CSS Syntax

```
selector {  
  property: value;  
  property: value;  
}  
  
/* Example */  
h1 {  
  color: blue;  
  font-size: 36px;  
  text-align: center;  
}
```

Three parts:

- Selector — Which element to style (e.g., `h1`, `p`, `.class`, `#id`).
- Property — What to change (e.g., `color`, `font-size`).
- Value — What to set it to (e.g., `blue`, `36px`).

3. Ways to Add CSS

Inline CSS

Written directly in the HTML element. Lowest priority. Not recommended for large projects.

```
<p style="color: red; font-size: 20px;">This is red text.</p>
```

Embedded (Internal) CSS

Written inside a `<style>` tag in the `<head>`.

```
<head>
  <style>
    p {
      color: blue;
      font-size: 18px;
    }
  </style>
</head>
```

External CSS

Written in a separate `.css` file and linked to the HTML. This is the best and most organised approach.

```
<!-- In the HTML <head> -->
<link rel="stylesheet" href="styles.css">

/* In styles.css */
body {
  font-family: Arial, sans-serif;
  background-color: #f0f0f0;
}
```

WORKOUT

1. Create a new file called `styles.css` in your project folder.
2. Link it to your `index.html` using `<link>`.
3. Set the body background-color to a light grey (`#f5f5f5`).
4. Set all `h1` elements to use color: navy and font-size: 40px.
5. Set all paragraphs to font-size: 18px and line-height: 1.6.

CSS Selectors

2. CSS Selectors

Selectors tell CSS which elements to style.

```
/* Type Selector – targets all <p> elements */
p { color: black; }

/* Universal Selector – targets everything */
* { margin: 0; padding: 0; box-sizing: border-box; }

/* Class Selector – targets elements with class="highlight" */
.highlight { background-color: yellow; }

/* ID Selector – targets element with id="header" */
#header { background-color: navy; color: white; }

/* Descendant Selector – targets <a> inside <nav> */
nav a { color: white; text-decoration: none; }

/* Child Selector – direct children only */
ul > li { list-style: square; }

/* Attribute Selector */
input[type="email"] { border: 2px solid blue; }
```



WORKOUT

1. Create 5 paragraphs. Give 3 of them class="featured".
2. Use a class selector to make featured paragraphs have a blue left border.
3. Give one paragraph id="special" and make it have a gold background.
4. Use the universal selector to remove all default margin and padding.
5. BONUS: Use an attribute selector to style all required form inputs.

3. CSS Colours

```
/* Named colours */
color: red;
color: navy;
```

```
color: tomato;

/* Hex codes */
color: #FF0000; /* red */
color: #1A3C5E; /* dark navy */

/* RGB values */
color: rgb(255, 0, 0); /* red */
color: rgba(255, 0, 0, 0.5); /* semi-transparent red */

/* HSL (Hue, Saturation, Lightness) */
color: hsl(0, 100%, 50%); /* red */
```

4. CSS Units

CSS uses different units for measurement:

- px — Pixels. Fixed size. (e.g., font-size: 16px)
- % — Percentage of the parent element. (e.g., width: 50%)
- em — Relative to the current element's font size. (e.g., padding: 1em)
- rem — Relative to the root (html) font size. More predictable than em.
- vw / vh — Percentage of the viewport width/height. (e.g., width: 100vw)

5. CSS Box Model

Every HTML element is a box. The box model describes the space around content.

```
div {
  width: 300px; /* content width */
  padding: 20px; /* space inside the border */
  border: 2px solid black; /* the border */
  margin: 30px; /* space outside the border */
}

/* box-sizing: border-box makes width include padding and border */
* {
  box-sizing: border-box;
}
```



TIP

Always set box-sizing: border-box on all elements (*).

Without it, padding and border ADD to your element's width — confusing!

With it, width: 300px always means exactly 300px total.

6. Fonts and Text

```
body {
  font-family: Arial, Helvetica, sans-serif;
  font-size: 16px;
  font-weight: normal; /* or bold, 100-900 */
  font-style: normal; /* or italic */
  line-height: 1.6;
}

h1 {
  font-size: 2.5rem;
  font-weight: 700;
  letter-spacing: 2px;
  text-transform: uppercase;
  text-align: center;
}

p {
  text-indent: 20px;
  word-spacing: 5px;
  text-decoration: underline;
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}
```

7. Backgrounds

```
body {
  background-color: #f5f5f5;
}

.hero {
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  background-attachment: fixed; /* parallax effect */
}
```

```
/* Gradient backgrounds */
.gradient {
  background: linear-gradient(to right, #1A3C5E, #2E86C1);
}
```

WORKOUT

1. Create a hero section div with a full-width background image.
2. Set background-size: cover and background-position: center.
3. Create a footer with a dark linear gradient background.
4. Style headings with letter-spacing and text-transform: uppercase.
5. Add text-shadow to your h1.

8. Display, Flexbox, and Grid

Display Property

```
display: block;      /* full width, starts on new line */
display: inline;     /* only as wide as content, no width/height */
display: inline-block; /* like inline but can set width/height */
display: none;       /* hides element completely */
```

Flexbox

```
.container {
  display: flex;
  flex-direction: row;      /* row (default) or column */
  justify-content: center;  /* horizontal alignment */
  align-items: center;      /* vertical alignment */
  gap: 20px;               /* space between items */
  flex-wrap: wrap;         /* wrap to next line if needed */
}

.item {
  flex: 1; /* grow and shrink equally */
}
```

CSS Grid

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
}
```

```
grid-template-rows: auto;
gap: 20px;
}

.grid-item {
  grid-column: span 2; /* span across 2 columns */
}
```

WORKOUT

1. Build a navigation bar using Flexbox.
 - Logo on the left, nav links on the right.
 - Use justify-content: space-between.
2. Build a 3-column card layout using CSS Grid.
 - Each card has an image, title, and description.
 - Use gap: 24px between cards.
3. BONUS: Make the grid responsive — 1 column on mobile, 3 on desktop.

9. Positioning

```
/* Relative – offset from normal position */
.box {
  position: relative;
  top: 20px;
  left: 10px;
}

/* Absolute – positioned relative to nearest positioned ancestor */
.badge {
  position: absolute;
  top: 0;
  right: 0;
}

/* Fixed – stays in place when scrolling */
.navbar {
  position: fixed;
  top: 0;
  width: 100%;
}
```

10. CSS Animations and Transitions

```
/* Transition – smooth change between states */
button {
  background-color: blue;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: darkblue;
}

/* Animation with @keyframes */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(-20px); }
  to   { opacity: 1; transform: translateY(0); }
}

.hero-title {
  animation: fadeIn 1s ease forwards;
}
```

11. CSS Variables

```
/* Define variables on the root element */
:root {
  --primary-color: #1A3C5E;
  --accent-color: #2E86C1;
  --font-size-base: 16px;
  --border-radius: 8px;
}

/* Use them anywhere */
button {
  background-color: var(--primary-color);
  border-radius: var(--border-radius);
  font-size: var(--font-size-base);
}

a {
  color: var(--accent-color);
}
```

```
}
```

**TIP**

CSS Variables make it easy to update your entire colour scheme in one place. Change `--primary-color` once and every element using it updates automatically. This is essential for building professional, maintainable websites.

12. Responsive Web Design

```
/* Mobile-first approach */

/* Base styles (mobile) */
.container {
  width: 100%;
  padding: 0 16px;
}

/* Tablet (768px and above) */
@media (min-width: 768px) {
  .container {
    max-width: 768px;
    margin: 0 auto;
  }
}

/* Desktop (1024px and above) */
@media (min-width: 1024px) {
  .container {
    max-width: 1200px;
  }
}
```

**WORKOUT**

1. Build a full webpage with:
 - A fixed navbar with logo and links
 - A hero section with a gradient background and centered text
 - A 3-column services section using CSS Grid
 - A contact form
 - A footer

2. Use CSS Variables for all your colours.
3. Add hover transitions to all buttons and links.
4. Make the layout responsive using media queries.

MODULE 3

MODULE 3: Bootstrap

Bootstrap is a free, open-source CSS framework that helps you build responsive, professional-looking websites faster. Instead of writing all your CSS from scratch, Bootstrap provides ready-made components — navbars, buttons, cards, modals, and more.

1. Getting Started with Bootstrap

Add Bootstrap to your project via CDN (no installation needed):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bootstrap Page</title>
  <!-- Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>

  <h1 class="text-center text-primary">Hello Bootstrap!</h1>

  <!-- Bootstrap JS (for interactive components) -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>
</body>
</html>
```

2. Bootstrap Grid System

Bootstrap uses a 12-column grid system. You decide how many columns each section takes up.

```
<div class="container">
  <div class="row">

    <!-- Full width on mobile, half width on medium screens, 1/3 on large -->
    <div class="col-12 col-md-6 col-lg-4">Column 1</div>
    <div class="col-12 col-md-6 col-lg-4">Column 2</div>
```

```
<div class="col-12 col-md-12 col-lg-4">Column 3</div>

</div>
</div>
```

Breakpoints:

- (none) — xs: All screen sizes (< 576px)
- sm — Small: 576px and up
- md — Medium: 768px and up
- lg — Large: 992px and up
- xl — Extra large: 1200px and up

3. Bootstrap Typography

```
<h1 class="display-1">Giant Heading</h1>
<h1 class="display-4">Slightly Smaller</h1>

<p class="lead">This is a lead paragraph – slightly larger.</p>
<p class="text-muted">This is muted/grey text.</p>
<p class="text-primary">Primary colour text.</p>
<p class="text-success">Green / success text.</p>
<p class="text-danger">Red / error text.</p>

<p class="text-center">Centered text.</p>
<p class="text-uppercase fw-bold">Bold uppercase.</p>
```

4. Bootstrap Buttons

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-light">Light</button>
<button class="btn btn-dark">Dark</button>

<!-- Outline buttons -->
<button class="btn btn-outline-primary">Outline Primary</button>
```

```
<!-- Button sizes -->
<button class="btn btn-primary btn-lg">Large</button>
<button class="btn btn-primary btn-sm">Small</button>

<!-- Full width button -->
<button class="btn btn-primary w-100">Full Width</button>
```

5. Bootstrap Tables

```
<table class="table table-striped table-hover table-bordered">
  <thead class="table-dark">
    <tr>
      <th>Name</th>
      <th>Course</th>
      <th>Score</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Amaka</td>
      <td>Web Dev</td>
      <td>92</td>
    </tr>
  </tbody>
</table>

<!-- Responsive table (scrolls on small screens) -->
<div class="table-responsive">
  <table class="table">...</table>
</div>
```

6. Bootstrap Alerts

```
<div class="alert alert-success" role="alert">
  Registration successful! Welcome aboard.
</div>

<div class="alert alert-danger" role="alert">
  Error: Please check your email and password.
</div>

<div class="alert alert-warning alert-dismissible fade show" role="alert">
```

```
Warning: Your session will expire soon.  
<button type="button" class="btn-close" data-bs-dismiss="alert"></button>  
</div>
```

7. Bootstrap Navigation Bar

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">  
  <div class="container">  
    <a class="navbar-brand" href="#">MyApp</a>  
  
    <!-- Mobile toggle button -->  
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"  
      data-bs-target="#navMenu">  
      <span class="navbar-toggler-icon"></span>  
    </button>  
  
    <!-- Nav links -->  
    <div class="collapse navbar-collapse" id="navMenu">  
      <ul class="navbar-nav ms-auto">  
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>  
        <li class="nav-item"><a class="nav-link" href="#">About</a></li>  
        <li class="nav-item"><a class="nav-link" href="#">Services</a></li>  
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>  
      </ul>  
    </div>  
  </div>  
</nav>
```

8. Bootstrap Cards

```
<div class="row row-cols-1 row-cols-md-3 g-4">  
  
  <div class="col">  
    <div class="card h-100">  
        
      <div class="card-body">  
        <h5 class="card-title">HTML Basics</h5>  
        <p class="card-text">Learn the building blocks of the web.</p>  
        <a href="#" class="btn btn-primary">Enroll Now</a>  
      </div>  
    </div>  
  </div>  
</div>
```

```
</div>
```

9. Bootstrap Forms

```
<form>
  <div class="mb-3">
    <label for="email" class="form-label">Email address</label>
    <input type="email" class="form-control" id="email"
      placeholder="name@example.com">
  </div>

  <div class="mb-3">
    <label for="password" class="form-label">Password</label>
    <input type="password" class="form-control" id="password">
  </div>

  <div class="mb-3 form-check">
    <input type="checkbox" class="form-check-input" id="remember">
    <label class="form-check-label" for="remember">Remember me</label>
  </div>

  <button type="submit" class="btn btn-primary w-100">Login</button>
</form>
```



WORKOUT

1. Build a complete Bootstrap webpage for a fictional school:
 - Dark navbar with school name and Home/About/Courses/Contact links.
 - Hero section with display heading, subtitle, and two CTA buttons.
 - 3-column course cards (HTML, CSS, JavaScript) using Bootstrap grid.
 - Responsive table showing student names and grades.
 - A contact form using Bootstrap form classes.
 - A dismissible alert at the top: "Admissions now open!".
 - A dark footer.
2. The page must be fully responsive (test on mobile size in browser DevTools).

MODULE 4

MODULE 4: JavaScript

JavaScript (JS) is the programming language of the web. While HTML structures content and CSS styles it, JavaScript makes it interactive and dynamic.

- Click a button and something happens → JavaScript
- A form validates before submitting → JavaScript
- Content updates without refreshing the page → JavaScript

Module 1: Writing Your First JavaScript Program

How to Add JavaScript to a Page

```
<!-- Option 1: Inline (not recommended) -->
<button onclick="alert('Hello!')">Click Me</button>

<!-- Option 2: Internal <script> tag -->
<script>
  console.log("Hello from JavaScript!");
</script>

<!-- Option 3: External file (best practice) -->
<script src="app.js"></script> <!-- Place before </body> -->
```

```
// Your first JavaScript program (in app.js)
console.log("Hello, World!");
alert("Welcome to JavaScript!");
```



WORKOUT

1. Create app.js and link it to your HTML page.
2. Write console.log("My name is [your name]") and open the browser console.
3. Write an alert() that says "Welcome to my page!".
4. Open DevTools (F12) > Console tab to see your output.

Module 2: The Grammar of JavaScript

Variables

```
// Three ways to declare variables
var name = "Amaka";           // Old way (avoid)
let age = 22;                  // Modern, can be changed
const country = "Nigeria";    // Cannot be reassigned

// Data types
let text = "Hello";           // String
let number = 42;               // Number
let decimal = 3.14;           // Number (float)
let isStudent = true;         // Boolean
let nothing = null;           // Null
let unknown;                   // Undefined
```

Operators

```
// Arithmetic
let sum = 10 + 5;             // 15
let diff = 10 - 5;            // 5
let product = 10 * 5;         // 50
let quotient = 10 / 5;        // 2
let remainder = 10 % 3;       // 1

// Comparison
5 == "5"    // true (loose, ignores type)
5 === "5"   // false (strict, checks type)
10 > 5       // true
10 !== 5     // true

// Logical
true && false // AND → false
true || false // OR  → true
!true         // NOT → false

// String concatenation
let greeting = "Hello, " + "World!"; // "Hello, World!"
let message = `Welcome, ${name}!`;    // Template literal
```

Functions

```
// Regular function declaration
```

```
function greet(name) {
  return "Hello, " + name + "!";
}
console.log(greet("Emeka")); // "Hello, Emeka!"

// Arrow function (modern shorthand)
const add = (a, b) => a + b;
console.log(add(5, 3)); // 8

// Function with default parameter
function welcome(name = "Guest") {
  return `Welcome, ${name}!`;
}
console.log(welcome()); // "Welcome, Guest!"
console.log(welcome("Chioma")); // "Welcome, Chioma!"
```



WORKOUT

1. Create variables for: your name, age, city, and whether you are a student.
2. Log them all to the console using template literals.
3. Write a function called `calculateArea(width, height)` that returns `width * height`.
4. Write a function called `gradeScore(score)` that returns A (90+), B (80+), C (70+), F (below 70).
5. Test both functions with different inputs.

Module 3: Logic and Control

Conditional Statements

```
// if / else if / else
let score = 75;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```



```
// Ternary operator (shorthand if/else)
let status = age >= 18 ? "Adult" : "Minor";

// Switch statement
switch (day) {
  case "Monday":
    console.log("Start of the week!");
    break;
  case "Friday":
    console.log("TGIF!");
    break;
  default:
    console.log("Regular day");
}
```

Loops

```
// for loop
for (let i = 1; i <= 5; i++) {
  console.log("Count: " + i);
}

// while loop
let count = 0;
while (count < 5) {
  console.log("While: " + count);
  count++;
}

// for...of (arrays)
const fruits = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
  console.log(fruit);
}

// forEach (array method)
fruits.forEach(fruit => console.log(fruit));
```



WORKOUT

1. Use a for loop to print the numbers 1 to 10.
2. Use a for loop to print only even numbers between 1 and 20.
3. Create an array of 5 student names.

4. Use `forEach` to print each name with "Student: " in front.
5. Write an `if/else` to check if a number is positive, negative, or zero.

Module 4: Built-In Methods — Strings, Numbers, Dates

```
// String methods
let str = "Hello, World!";

str.length           // 13
str.toUpperCase()     // "HELLO, WORLD!"
str.toLowerCase()     // "hello, world!"
str.includes("World") // true
str.replace("World", "Nigeria") // "Hello, Nigeria!"
str.split(", ")       // ["Hello", "World!"]
str.trim()             // removes leading/trailing spaces
str.slice(0, 5)        // "Hello"

// Number methods
let n = 3.14159;
n.toFixed(2)          // "3.14"
Math.round(3.7)       // 4
Math.floor(3.9)       // 3
Math.ceil(3.1)        // 4
Math.max(1, 5, 3)     // 5
Math.min(1, 5, 3)     // 1
Math.random()         // random between 0 and 1

// Dates
let now = new Date();
now.getFullYear()     // e.g., 2025
now.getMonth()        // 0-11 (0 = January)
now.getDate()         // day of month
now.toLocaleDateString() // "6/16/2025"
```



WORKOUT

1. Take a full name string and output it in ALL CAPS.
2. Check if a string contains the word "Nigeria".
3. Generate a random number between 1 and 100.
4. Write a function that returns today's date formatted as "Day, DD Month YYYY".
5. BONUS: Take a sentence and count how many words it has.

Module 5: DOM Manipulation

The DOM (Document Object Model) is the browser's representation of your HTML page. JavaScript can read and change it.

```
// Selecting elements
document.getElementById("myId")
document.querySelector(".myClass")      // first match
document.querySelectorAll("p")          // all <p> elements

// Changing content
let heading = document.querySelector("h1");
heading.textContent = "New Heading!";
heading.innerHTML = "<em>Italic Heading</em>";

// Changing styles
heading.style.color = "red";
heading.style.fontSize = "3rem";

// Adding/removing classes
heading.classList.add("active");
heading.classList.remove("active");
heading.classList.toggle("dark-mode");

// Reading/setting attributes
let img = document.querySelector("img");
img.getAttribute("src");
img.setAttribute("alt", "New description");

// Creating new elements
let newPara = document.createElement("p");
newPara.textContent = "I was created by JavaScript!";
document.body.appendChild(newPara);
```

WORKOUT

1. Create a page with an h1 and a button.
2. When the button is clicked, change the h1 text to "Button was clicked!".
3. Each click should also toggle the background colour of the page.
4. Create a button that adds a new <p> to the page each time it is clicked.
5. BONUS: Add a "Clear All" button that removes all added paragraphs.

Module 6: Events

```
// addEventListener is the recommended way to handle events
let button = document.querySelector("button");

button.addEventListener("click", function() {
  console.log("Button clicked!");
});

// Arrow function syntax
button.addEventListener("click", () => {
  alert("Clicked!");
});

// Common events
element.addEventListener("click", handler);
element.addEventListener("mouseover", handler);
element.addEventListener("mouseout", handler);
element.addEventListener("keydown", handler);
element.addEventListener("submit", handler);
element.addEventListener("change", handler);
element.addEventListener("input", handler);

// Event object
input.addEventListener("keydown", (event) => {
  console.log("Key pressed:", event.key);
});

// Prevent default behaviour (e.g., stop form submit)
form.addEventListener("submit", (event) => {
  event.preventDefault();
  console.log("Form submitted via JS!");
});
```

Module 7: Local Storage

```
// Save data
localStorage.setItem("username", "Amaka");
localStorage.setItem("theme", "dark");

// Retrieve data
```

```
let user = localStorage.getItem("username"); // "Amaka"

// Save objects (must convert to string)
let student = { name: "Emeka", score: 92 };
localStorage.setItem("student", JSON.stringify(student));

// Retrieve and convert back
let saved = JSON.parse(localStorage.getItem("student"));
console.log(saved.name); // "Emeka"

// Remove data
localStorage.removeItem("username");

// Clear all
localStorage.clear();
```

WORKOUT

1. Build a "Remember Me" feature:
 - A text input where the user types their name.
 - A "Save" button that stores it in localStorage.
 - When the page loads, check localStorage and greet the user by name.
2. Build a simple dark/light mode toggle:
 - A button that switches the background between dark and light.
 - Save the user's preference in localStorage.
 - When the page reloads, apply the saved preference automatically.

Module 8: Form Validation with JavaScript

```
<form id="registerForm">
  <input type="text" id="username" placeholder="Username">
  <span id="usernameError" style="color:red;"></span>

  <input type="email" id="email" placeholder="Email">
  <span id="emailError" style="color:red;"></span>

  <button type="submit">Register</button>
</form>

<script>
```

```
const form = document.getElementById("registerForm");

form.addEventListener("submit", (e) => {
  e.preventDefault();
  let valid = true;

  const username = document.getElementById("username").value.trim();
  const email = document.getElementById("email").value.trim();

  if (username.length < 3) {
    document.getElementById("usernameError").textContent =
      "Username must be at least 3 characters.";
    valid = false;
  } else {
    document.getElementById("usernameError").textContent = "";
  }

  if (!email.includes("@")) {
    document.getElementById("emailError").textContent =
      "Please enter a valid email.";
    valid = false;
  } else {
    document.getElementById("emailError").textContent = "";
  }

  if (valid) {
    alert("Registration successful!");
  }
});
</script>
```

Module 9: Introduction to Ajax and JSON

```
// JSON – JavaScript Object Notation
// Used to send and receive data

let student = {
  "name": "Amaka",
  "age": 22,
  "courses": ["HTML", "CSS", "JavaScript"]
};

// Fetch API – get data from a server
```

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
    data.forEach(user => {
      console.log(user.name);
    });
  })
  .catch(error => {
    console.error("Error:", error);
  });

// Async/await syntax (modern, cleaner)
async function getUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error("Failed to fetch:", error);
  }
}

getUsers();
```



WORKOUT

1. Use the fetch API to get data from: <https://jsonplaceholder.typicode.com/posts>
2. Display the first 10 posts on your page.
3. Each post should show: title (as `<h3>`) and body (as `<p>`).
4. Add a "Load Posts" button that triggers the fetch.
5. BONUS: Add a loading spinner that shows while data is being fetched.

Module 10: AI-Assisted Web Development

Modern developers use AI tools to write, debug, and improve their code faster. This is a professional skill in today's industry.

Working with AI Code Agents

- Describe what you want to build in plain English — the AI writes the code.
- Always review AI-generated code — understand it before using it.

- AI is a co-pilot, not a replacement for understanding.

Debugging with AI

```
// Paste your error message and broken code into an AI chat
// Example prompt:
// "This JavaScript code throws 'TypeError: Cannot read property
// of undefined'. Here is my code: [paste code].
// What is wrong and how do I fix it?"
```

Vibe Coding

Vibe coding is describing your project idea conversationally to an AI and having it generate the initial structure, components, and logic for you to build on.



TIP

Good prompt example: "Build me a student registration form with name, email, phone, and course fields. Validate all fields before submission. Show error messages under each invalid field in red. Use Bootstrap for styling."

Then review, modify, and customise the output to fit your needs.



WORKOUT

1. Use an AI assistant (ChatGPT, Claude, Copilot, etc.) to generate:
 - A full registration form with validation.
 - Review every line of the code and add comments explaining what each part does.
2. Use AI to debug a broken piece of code (introduce a bug on purpose, then ask AI to find it).
3. Write a prompt to an AI to build a "To-Do List" app with:
 - Add tasks, mark complete, delete tasks.
 - Save tasks to localStorage.
 - Then customise the result with your own styles and logic.

FINAL PROJECT: Build Your Portfolio Website

This is your capstone project. Apply everything you have learned — HTML, CSS, Bootstrap, and JavaScript — to build a personal portfolio website.

Project Requirements

Pages Required:

7. index.html — Home page
8. about.html — About me page
9. projects.html — My projects page
10. contact.html — Contact form page

Technical Requirements:

- HTML: Semantic tags (header, nav, main, section, footer), forms with validation.
- CSS: Custom stylesheet with variables, flexbox/grid layout, animations, responsive design.
- Bootstrap: Navbar, cards, buttons, responsive grid.
- JavaScript: Form validation, DOM manipulation, localStorage for theme preference, fetch API.

Content Requirements:

- Your name, photo, and a short bio.
- A skills section (HTML, CSS, JavaScript, etc.).
- At least 2 project cards with description and links.
- A working contact form with JS validation.
- Dark/Light mode toggle saved to localStorage.

Grading Criteria:

Criteria	Points
HTML Structure & Semantics	20 points
CSS Styling & Layout	20 points
Bootstrap Integration	15 points

JavaScript Functionality	25 points
Responsiveness (Mobile)	10 points
Design & Creativity	10 points

Good luck, and build something great! 🚀